

Near-Frontier with Reset (NFR) Planner: Formal Overview

ARC-AGI Abstraction Navigator Notes

November 6, 2025

1 Problem Setting

Let $\mathcal{G} = (S, A, T)$ denote the agent interaction graph. States $s \in S$ encode masked ARC frames, actions $a \in A$ are constrained to the four arrows, and $T : S \times A \rightarrow S \cup \{\perp\}$ is the partial transition function estimated from experience ($\perp$ indicates an unknown successor).

Each play starts at a known level start state $s_0 \in S$. The planner is given:

- $S_{\text{disc}} \subseteq S$: discovered states from prior episodes.
- $T_{\text{known}} : S_{\text{disc}} \times A \rightarrow S_{\text{disc}} \cup \{\perp\}$: the partial transition map recovered from experience (returning $\perp$ when a transition has not yet been seen).
- $A(s) \subseteq A$: actions the environment currently allows at state s .
- $\mathcal{F} = \{s \in S_{\text{disc}} \mid \exists a \in A(s) : T_{\text{known}}(s, a) = \perp\}$, the *frontier* states whose outgoing edges are incompletely observed.

During an episode the agent sits in state s_t with available actions $A_t = A(s_t)$. The planner must output $a_t \in A_t \cup \{\text{RESET}\}$ to minimise the distance to a frontier while optionally using reset jumps to s_0 .

2 Algorithm

2.1 Definition

Given the current state s_t , the available actions $A_t = A(s_t)$, and the level start state s_0 , the planner executes:

1. Run breadth-first search on the known subgraph $\mathcal{G}_t = (S_{\text{disc}}, A, T_{\text{known}})$ from both s_t and s_0 :

$$D_t^c, P_t^c = \text{BFS}(\mathcal{G}_t, s_t), \quad (1)$$

$$D_t^0, P_t^0 = \text{BFS}(\mathcal{G}_t, s_0). \quad (2)$$

Here D_t^c and D_t^0 record shortest-path distances, while the predecessor maps P_t^c and P_t^0 associate each reachable vertex x with the pair $(\text{parent}(x), \text{action}(x))$ describing the final step on the corresponding shortest path.

2. Compute the reset-augmented cost for every frontier $f \in \mathcal{F}$:

$$J(f) = \min\{D_t^c(f), 1 + D_t^0(f)\}. \quad (3)$$

3. Select the frontier

$$f^* = \arg \min_{f \in \mathcal{F}} (J(f), D_t^0(f), f),$$

using lexicographic order on the triple $(J(f), D_t^0(f), f)$.

4. If $J(f^*)$ is finite and A_t is non-empty, return

$$a_t = \text{first edge on the path recovered from } P_t^c \text{ between } s_t \text{ and } f^*$$

when $D_t^c(f^*) \leq 1 + D_t^0(f^*)$, and return RESET otherwise.

When $J(f^*)$ is infinite (no reachable frontier) or A_t is empty, the procedure returns no action.

2.2 Implementation Notes

The code performs the following additional checks:

- If the frontier loop in step 2 yields no finite-cost candidate, the planner abstains and the caller falls back to its default policy.
- If the chosen frontier satisfies $s_t = f^*$ or the predecessor map does not contain a route, the planner probes the first $a \in A_t$ with $T_{\text{known}}(s_t, a) = \perp$; it abstains when every available action is already known.
- The RESET branch is triggered only when the inequality $D_t^c(f^*) > 1 + D_t^0(f^*)$ holds, i.e. restarting from s_0 plus one action is strictly cheaper than moving directly from s_t .

3 Desirable Properties

Completeness on the Known Subgraph. As long as $\mathcal{G}_t$ is connected between s_t and all frontiers, BFS ensures $D_t^c(f)$ equals the true shortest distance in $\mathcal{G}_t$. Therefore the navigation step reaches f^* in minimal steps without reset.

Reset Optimality. The cost (3) chooses between continuing from s_t or incurring one reset plus the path from s_0 . This is optimal under the assumption that resetting teleports to s_0 in one step and costs a unit time penalty.

Frontier Prioritisation. Any frontier with strictly smaller $J(f)$ than all others will eventually be chosen. If exploration uncovers new edges, J automatically updates without retuning.

Idempotence of Known Actions. The planner never replays an action whose successor is already in $T_{\text{known}}(s_t, a)$ when probing frontiers; thus exploration focuses on unknown transitions.